

25. Observability / Log Ingestion at Org Scale

Interviewer: Design the logging backend for a large company — **thousands of microservices**, each emitting logs continuously. Engineers need to search these logs ("show me all ERROR logs for the checkout service in the last 15 minutes with this trace-id"), build dashboards, and get alerted. It has to handle the firehose without falling over and without slowing down the services doing the logging. Go.

Candidate: This is a classic **write-heavy ingest + read-flexible search** problem, and the trap is treating it like a normal database. The volume is enormous, the data is write-once/read-rarely, and — critically — **logging must never slow down or crash the services producing the logs**. Let me clarify before designing.

1. Clarifying the requirements

Candidate: 1. Roughly how many services and what log volume — MB/s? millions of lines/sec? 2. What's the primary read pattern — **full-text search** ("find this error string"), **filtered lookup** (by service + level + time + trace-id), or **aggregation** ("error rate over time")? These pull toward very different storage. 3. Retention — hot-searchable for how long, then archived or deleted? 4. Is this just logs, or also **metrics and traces**? (The "three pillars" of observability — I'll focus on logs but note where metrics/traces differ.) 5. How fresh must search be — is a few seconds of ingest lag acceptable?

Interviewer: ~5,000 services, peak **~5 million log lines/sec**, avg 500 bytes each. Reads are mostly **filtered search** by service/level/time/trace-id, plus some full-text and some aggregation for dashboards. Hot-searchable for 7 days, then archived to cheap storage for 1 year. Focus on logs. A **few seconds of ingest lag is fine**.

Candidate: Great — that "few seconds lag is fine" is a gift, it lets me make the whole pipeline **asynchronous and buffered**. Requirements:

Functional - Services emit structured logs (JSON: `{ts, service, level, host, trace_id, msg, ...}`). - Engineers search/filter by service + level + time range + trace_id, some full-text. - Dashboards aggregate (error rate, volume by service). - Alerting on log patterns (e.g. "ERROR rate > X"). - 7-day hot retention, 1-year cold archive.

Non-functional - **Ingest must be decoupled and non-blocking** — a slow/broken logging backend must NOT stall or crash the 5,000 services. This is the #1 rule of observability infra. - **Massive write throughput** (5M lines/s $\approx$ 2.5 GB/s $\approx$ ~216 TB/day raw). - **Read latency** in the low seconds for filtered search over the hot window. - **Cost efficiency** — you cannot keep 216 TB/day in an expensive hot store; tiering is mandatory. - **Lossy-tolerant**: logs are best-effort. Dropping a tiny fraction under extreme load is acceptable (unlike, say, payments). This shapes the backpressure strategy.

2. Estimation — the numbers force the architecture

5M lines/sec * 500 bytes = 2.5 GB/sec ingest
= 150 GB/min = ~9 TB/hour = ~216 TB/day (raw, uncompressed)

Logs compress well (~10:1 with columnar + dictionary) -> ~21 TB/day stored hot.
7-day hot window: ~150 TB hot (searchable index).
1-year cold archive (compressed, in object storage): ~7.6 PB. Cheap blob, not indexed.

Read QPS: modest — maybe hundreds of engineer searches/min + dashboard refreshes.
The system is ~99.99% writes. So: optimize the write path relentlessly;
the read path can be heavier per-query as long as it's bounded.

=> Conclusions the numbers hand us:

- 2.5 GB/s cannot hit a database synchronously -> a buffer (Kafka) MUST sit in front.
- 216 TB/day cannot all stay hot -> tiered storage (hot index + cold blob) is required.
- Writes >> reads -> a write-optimized store (LSM / columnar), not a b-tree OLTP DB.

3. The V1 and why the naive version dies

Candidate: Let me state the naive version so we agree on why it fails.

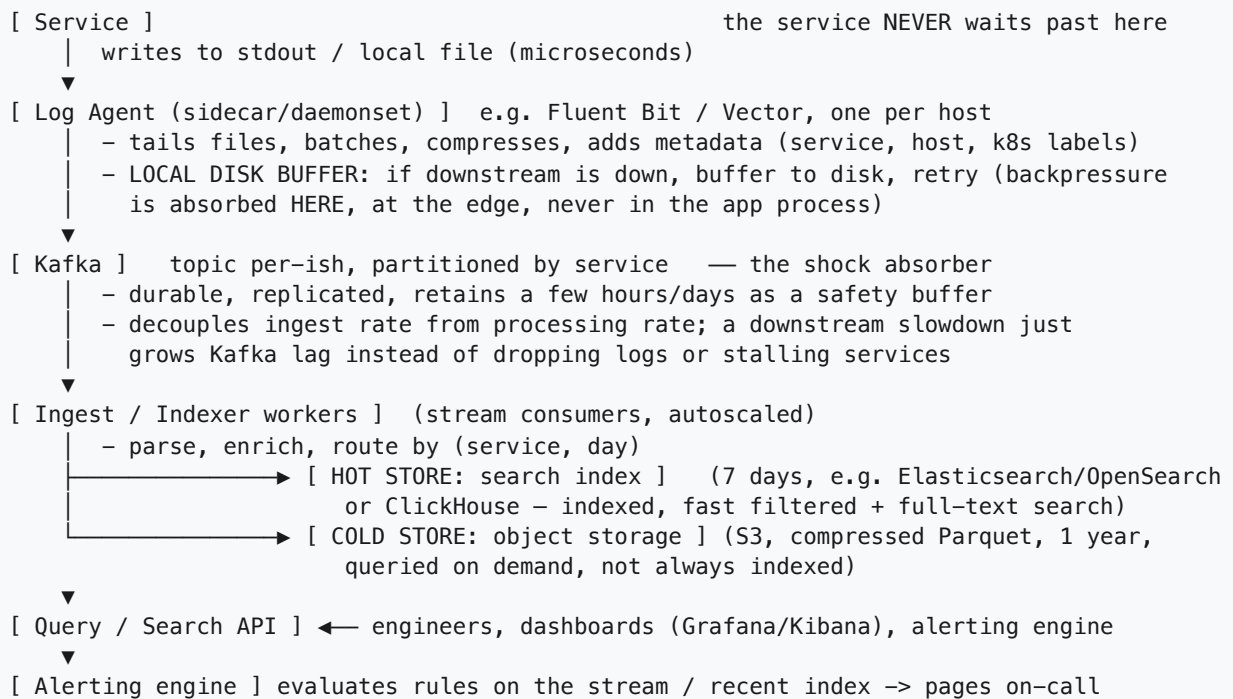
```
[ 5,000 services ] --HTTP POST each log line--> [ Log API ] --INSERT--> [ Postgres ]
```

Why it dies three ways: 1. **Synchronous coupling.** If Postgres is slow or down, every service's log call blocks or errors — logging backpressure crashes production. **Cardinal sin.** 2. **Write throughput.** Postgres does maybe tens of thousands of row inserts/sec on one node. We need 5,000,000/sec. Off by ~100x. 3. **Storage + query.** A b-tree relational DB indexing 216 TB/day and doing full-text `LIKE '%error%'` scans is hopeless — wrong tool entirely.

The fix touches every layer: **decouple the emit, buffer the firehose, write to a search-optimized tiered store, and tier aging data to cheap blob.**

4. The pipeline — the core of the design

Candidate: Here's the shape. The golden rule: **the service hands its log to a local agent and immediately moves on.** Everything after that is asynchronous.



Why each stage exists: - **Local agent + disk buffer:** the *only* place backpressure is allowed to live. The app writes to a file/stdout at memory speed and is done. If Kafka is unreachable, the agent buffers to local disk and retries — the app never knows. - **Kafka as the shock absorber:** converts a spiky 2.5 GB/s firehose into a durable stream the indexers consume at their own pace. A traffic spike or an indexer outage just increases **consumer lag** (which we alert on) — it never drops logs or touches the services. This is the single most important box in the design. - **Indexer workers** are stateless and autoscale on Kafka lag. - **Hot/cold split** is what makes the cost sane (see §6).

5. Which datastore — and why (the detailed part)

Interviewer: You waved at "search index" and "object storage." Be specific — what database, and why not the obvious alternatives?

Candidate: The read pattern (filtered lookups by service/level/time/trace-id + some full-text + some aggregation) plus write-once semantics points to a **search/analytics engine, not an OLTP database**. My default is **Elasticsearch/OpenSearch for the hot tier**, with **ClickHouse as a strong alternative** when the workload leans more analytical.

Concern	Choice	Why this, and why not the alternative
Edge collection	Fluent Bit / Vector agent (sidecar/daemonset)	Runs next to the service, tails stdout/files, batches + buffers to local disk. <i>Not direct HTTP from the app</i> — that couples app health to backend health.
Ingest buffer	Kafka	Durable, replicated, high-throughput partitioned log; absorbs the firehose and decouples produce rate from index rate; retains a replay window. <i>Not a synchronous write to the store</i> — one slow store would stall everything; <i>not SQS</i> — we want high-

Concern	Choice	Why this, and why not the alternative
		throughput ordered partitions + a retained replayable log we can re-index from.
Hot searchable store (7d)	Elasticsearch / OpenSearch	Inverted index gives fast full-text + filtered search; time-based indices make retention/rollover trivial (drop an index to expire a day); mature for log search (the "L" of ELK). <i>Not Postgres</i> — can't index 216 TB/day or do fast <code>%text%</code> . <i>Not a plain KV</i> — no full-text/aggregation.
(Alternative) hot store	ClickHouse	If reads are mostly filtered + aggregation rather than heavy full-text, a columnar OLAP DB is far cheaper and faster to scan, with excellent compression. Many modern log platforms (e.g. Grafana Loki's label model, or ClickHouse-backed systems) go this route. Choose ES for full-text-heavy, ClickHouse for aggregation/cost-heavy.
Cold archive (1yr)	Object storage (S3/GCS), compressed Parquet	Pennies/GB for 7.6 PB; queried on demand via Athena/Presto or re-hydrated to the hot store for investigations. <i>Not keeping it in ES</i> — indexing cost for rarely-read data is enormous.
Metadata / index catalog	A small relational DB or the store's own catalog	Tracks which indices/partitions exist, retention state — tiny, transactional.
Alerting rule state	Time-series / rules engine (e.g. on the stream or Prometheus-style)	Evaluate patterns on recent windows; separate from raw-log storage.

Say-it-out-loud justification: "A local agent with a disk buffer keeps logging non-blocking for the services; Kafka is the durable shock absorber that decouples a 2.5 GB/s firehose from the indexers; Elasticsearch (or ClickHouse for aggregation-heavy loads) is the write-optimized, time-partitioned hot store that makes filtered + full-text search fast for 7 days; and cheap object storage holds the 1-year archive at a hundredth the cost. A relational OLTP DB is the wrong tool on every axis — throughput, storage cost, and query type."

Why time-based indices matter: we roll a new index per service-day (or per hour). Search for "last 15 min" only touches today's index. Expiring 7-day-old data is a metadata delete of a whole index — no expensive row-by-row deletion.

6. Tiering + retention — the cost lever

age 0–7 days: HOT — Elasticsearch/ClickHouse, fully indexed, sub-second search
age 7–365 days: COLD — S3 Parquet, compressed ~10:1, queried via Athena/Presto on demand
age > 365 days: DELETE (or Glacier for compliance)

Index Lifecycle Management (ILM) automatically:
hot -> warm (fewer replicas, less RAM) -> cold (moved to S3) -> delete

Why: keeping everything hot would cost ~50–100× more. 99% of searches hit the last few hours; the long tail is rare investigations that can tolerate a slower cold query or a re-hydration step.

7. Consistency, latency, and backpressure

- **Consistency: eventual and lossy-tolerant.** A log is searchable a few seconds after it's emitted (Kafka → indexer → refresh interval). That's the "few seconds lag is fine" the interviewer granted. We do **not** need read-your-writes or transactions on logs.
- **Latency budget:** emit = microseconds (local write). Searchable = single-digit seconds. Query = low seconds for a filtered 7-day search (bounded by touching only relevant time-partitioned shards).
- **Backpressure under overload** (the key failure conversation): when ingest exceeds capacity, options in order of preference:
- **Buffer** — Kafka lag grows, indexers autoscale, catch up. First line of defense.
- **Shed by priority** — if lag is dangerous, drop `DEBUG / INFO` first, always keep `ERROR / FATAL` . Sampling high-volume noisy logs.
- **Never** apply backpressure to the application. Logs are best-effort; **dropping some DEBUG lines is strictly better than stalling production.** This is the philosophical core: observability must be a passive observer, never a source of outages.

8. Failure modes & graceful degradation

Failure	What happens	Mitigation
Elasticsearch cluster slow/down	New logs can't index	Kafka retains them; indexers replay on recovery — no loss, just delayed searchability
Kafka partition/broker down	Ingest at risk	Replication + multiple brokers; agents buffer to local disk during a full outage
Indexer can't keep up (lag grows)	Search falls behind real-time	Autoscale indexers on lag; shed DEBUG/INFO; alert on-call about the lag itself
A single service goes berserk (log storm)	One service floods the pipeline	Per-service quotas/rate-limits at the agent or Kafka quota; isolate its partition so it can't starve others
Object storage archival fails	Cold tier gaps	Retry; keep in warm tier until archived; monitor ILM
Cardinality/field explosion	Index mapping blows up	Cap dynamic fields; structured logging discipline; drop/route unknown fields

9. Follow-up curveballs

Interviewer: An engineer wants to trace one request across 30 services. How?

Candidate: Trace-id propagation. Each request gets a `trace_id` at the edge (distributed tracing, W3C Trace Context) that's passed through every service call and included in every log line. Searching `trace_id = X` in the hot index returns the full cross-service story in time order. This is where logs meet

tracing (the third pillar) — for deep latency breakdowns you'd add a tracing backend (Jaeger/Tempo) storing spans, but trace-id in logs covers the common case.

Interviewer: How do you stop one noisy service from drowning out everyone else?

Candidate: Multiple layers: **per-service Kafka quotas** and **agent-side rate limits/sampling** so a log storm is capped at the source; partition by service so one service's volume lives on its own partitions and can't starve others' consumers; and **priority-aware shedding** (drop that service's DEBUG first). Optionally per-team **budgets** with alerts when a service exceeds its expected volume.

Interviewer: Dashboards query "error count per service over the last 24h" and it's slow.

Candidate: Don't scan raw logs for that. **Pre-aggregate:** a stream job computes rollups (counts per service/level/minute) into a small time-series/OLAP table as logs flow. Dashboards read the tiny rollup, not 216 TB of raw lines. Raw logs are for *drilling in* after the aggregate points you at a spike. (This is exactly the metrics-vs-logs boundary — cheap numeric rollups for dashboards, raw logs for investigation.)

Interviewer: Compliance wants specific logs kept 7 years and provably immutable.

Candidate: Route those (by tag) to a separate **WORM** object-storage bucket (S3 Object Lock / immutability policy) with a 7-year retention lock, separate from the general 1-year archive. Keep them out of the hot tier entirely; they're for audit, not search.

10. Deep-dive: "Why not just push logs straight into Elasticsearch?"

Interviewer: You put Kafka in the middle. Why not have the agents write **directly to Elasticsearch**? One fewer moving part.

Candidate: Because Elasticsearch's indexing throughput and the incoming log rate are two different, independently-varying numbers — and the moment they diverge without a buffer between them, you either drop logs or stall the producers. Kafka is the shock absorber that decouples them.

```
Direct write (no buffer):
  agents —2.5 GB/s→ Elasticsearch
                        |
  ES slows during a merge / GC / hot shard
  -> back-pressure hits the AGENTS
  -> agents block or drop -> logs lost
```

With Kafka buffer:

```
agents → Kafka → indexers → ES
          (retains hours)  (pull at own pace)
```

ES slows -> Kafka lag grows ->
indexers catch up later ->
ZERO loss, producers unaffected

Concern	Direct agent → Elasticsearch	Agents → Kafka → indexers → ES
Rate mismatch	ES indexing rate must always $\geq$ incoming rate, or you drop/stall	Kafka absorbs bursts; indexers drain at their own pace
ES maintenance (merges, GC, rebalance)	Directly back-pressures producers → log loss or app stalls	Just grows consumer lag; invisible to producers
Reprocessing (new index, mapping change, second consumer)	Impossible — the data already went only to ES	Replay from the retained Kafka log any time
Fan-out to cold storage / SIEM / analytics	Each is another direct write from every agent	One Kafka topic, many independent consumers
Coupling	App/agent health tied to ES health	Fully decoupled

Why not skip both and log to the DB? A relational DB indexes far slower than ES and can't do fast full-text search over 216 TB/day — wrong tool on throughput *and* query type.

Rule of thumb: any time a fast, spiky producer feeds a slower or maintenance-prone consumer, put a durable log (Kafka) between them. The buffer converts "drop or stall" into "lag and recover."

11. Deep-dive: the Kafka buffer fills up during a prolonged outage

Interviewer: Elasticsearch is down for two hours during a traffic peak. Kafka is absorbing 2.5 GB/s of logs that nobody is draining. Walk me through what happens — Kafka isn't infinite.

Candidate: Right — the buffer buys time, not infinity. Let me trace it and show where it breaks.

```
t0 ES goes down. Indexers can't write -> they stop committing offsets -> Kafka lag climbs.
t0+ Producers unaffected: agents keep shipping, Kafka keeps accepting. Good so far.
2.5 GB/s * 3600s = ~9 TB/hour piling into Kafka's retained log.
If Kafka retention/disk = 12 TB, we have ~1.3 hours before the OLDEST data
starts getting deleted by retention -> THAT's where logs are actually lost,
silently, from the tail – not at the producer.
Also: agents' own local disk buffers were never stressed (Kafka stayed up), so the
risk here is Kafka retention expiry, not agent overflow.
```

So the failure isn't "Kafka rejects writes" — it's **retention silently dropping the oldest messages** once disk fills, and indexers falling so far behind that search is hours stale even after ES recovers. Ranked defenses:

1. **Size Kafka retention for the worst realistic outage, not the average.** If ops needs 4h to recover ES, provision disk for >4h of ingest. This is the first and cheapest lever.
2. **Autoscale indexers on lag, and let them burst on recovery.** When ES comes back, temporarily add indexer capacity to drain the backlog faster than real-time. Alert on consumer lag as a first-class SLO (it's your early-warning signal, long before loss).

3. **Priority tiers / shed low value first.** If disk is genuinely about to fill, drop `DEBUG` / `INFO` topics before `ERROR` / `FATAL` — route levels to separate topics so you can expire the cheap ones first. Never lose error logs to save debug logs.
4. **Spill to cheap storage.** A consumer can also stream the raw Kafka log to S3 continuously; even if ES indexing is behind, the data is durably in object storage and can be back-filled into the index later. Decouples *durability* from *searchability*.
5. **Tiered-storage Kafka.** Modern Kafka can offload older segments to object storage, making effective retention huge and cheap — the backlog lives on S3 instead of local broker disk.

The philosophy holds: **logs are best-effort, so under true exhaustion we shed the lowest-value data deliberately rather than stall production or lose errors** — but with retention sized for real recovery times plus S3 spill, actual loss should never happen.

What to say out loud: *"Kafka buys recovery time, not infinity — the real failure is retention silently dropping the tail once disk fills, and search going stale. I'd size retention for the worst-case ES recovery window, autoscale indexers to burst-drain on recovery, alert on consumer lag as an SLO, continuously spill raw logs to S3 so durability doesn't depend on ES, and if truly cornered, shed DEBUG before ERROR. Tiered-storage Kafka makes the backlog effectively unbounded and cheap."*

Summary — the mental model

Org-scale logging is a **"absorb a 2.5 GB/s write-once firehose without ever touching the health of the services producing it, then make 7 days of it searchable cheaply"** problem. The winning design: **non-blocking local agents with disk buffers** at the edge, **Kafka as the durable shock absorber** decoupling ingest from indexing, a **write-optimized, time-partitioned search store** (Elasticsearch for full-text, ClickHouse for aggregation) for the hot 7 days, and **cheap object storage** for the 1-year cold archive with lifecycle tiering. Consistency is eventual and best-effort; under overload you **shed low-priority logs, never stall production**. Logs are a passive observer of the system — the infrastructure must guarantee it can never become the cause of an outage.